

AI Application and Evaluation

Wilkins Exavier

Prompt: " Create a step-by-step guide on launching a virtual server using Amazon EC2. Explain it as if from the perspective of an AWS Cloud Engineer."

AI Output:

Overview

Here's how I'd **walk up** a new teammate through launching a virtual server (EC2 instance) in AWS—step by step, with the mindset of an AWS Cloud Engineer.

Step 1: Sign in and choose the right region

1. **Log in to the AWS Management Console.**
2. **Select the appropriate Region** in the top-right (e.g., *us-east-1*), ideally close to your users or aligned with project standards.

Step 2: Open EC2 and start the launch wizard

1. **Navigate to EC2:**
 - a. From the console, go to **Services** → **EC2**.
2. **Click "Launch instance."**
 - a. This opens the launch wizard where we define all the core settings for the virtual server.

Step 3: Name the instance and choose an AMI

1. **Give the instance a meaningful name** (e.g., *web-api-prod-01*).
2. **Select an Amazon Machine Image (AMI):**

- a. Common choices: **Amazon Linux 2**, Ubuntu, or a hardened custom AMI used by your team.
- b. For test workloads, I usually pick a Free Tier–eligible AMI like **Amazon Linux 2**.

Step 4: Choose an instance type

1. **Pick the instance family and size** based on workload:
 - a. For simple tests: `t2.micro` or `t3.micro` (often Free Tier–eligible).
 - b. For production: match CPU/RAM to app requirements (e.g., `m` for general purpose, `c` for compute).
2. **Confirm vCPU and memory** meet performance expectations.

Step 5: Configure key pair (SSH/RDP access)

1. **Create or select a key pair:**
 - a. If none exists, create a new key pair (e.g., `project-ec2-key`) and **download the .pem file** securely.
2. **Explain to the team:**
 - a. This private key is required to SSH into Linux instances or decrypt Windows admin passwords.

Step 6: Configure network settings (VPC, subnet, security group)

1. **VPC and subnet:**
 - a. Use the project's standard VPC and subnet (e.g., private subnet for backend, public for internet-facing).
2. **Assign public IP (if needed):**
 - a. Enable auto-assign public IP for internet-facing servers; disable for strictly internal workloads.
3. **Security group (firewall):**
 - a. Create or select a security group.
 - b. Add inbound rules only for required ports (e.g., 22 for SSH, 80/443 for web).
 - c. Keep it least-privilege—restrict source IP ranges to known admin networks or load balancers.

Step 7: Configure storage

1. Root volume:

- a. Choose an **EBS volume** type (commonly *gp2* or *gp3*) and size appropriate for OS and logs.

2. Additional data volumes (optional):

- a. Add separate volumes for application data, databases, or logs if required by the architecture.

Step 8: Review and launch

1. Review all settings:

- a. Name, AMI, instance type, key pair, VPC/subnet, security group, storage.

2. Confirm cost expectations (especially for non-Free Tier instances).

3. Click “Launch instance.”

- a. The instance state will move from **pending** to **running** once it’s ready.

Step 9: Connect to the instance

1. From the EC2 console, select the instance and click **Connect**.

2. Choose connection method:

- a. For Linux: SSH (using the `.pem` key) or **EC2 Instance Connect**.
- b. For Windows: RDP after retrieving the Administrator password.

3. Verify access and basic health:

- a. Check that you can log in, update packages, and reach any required endpoints.

Step 10: Post-launch best practices

1. Apply baseline configuration:

- a. OS updates, monitoring agents (CloudWatch), logging, and any hardening scripts your team uses.

2. Tag the instance:

- a. Add tags like Name, Environment, Owner, **Cost Center**, and Application for **Governance**.

3. Plan for lifecycle:

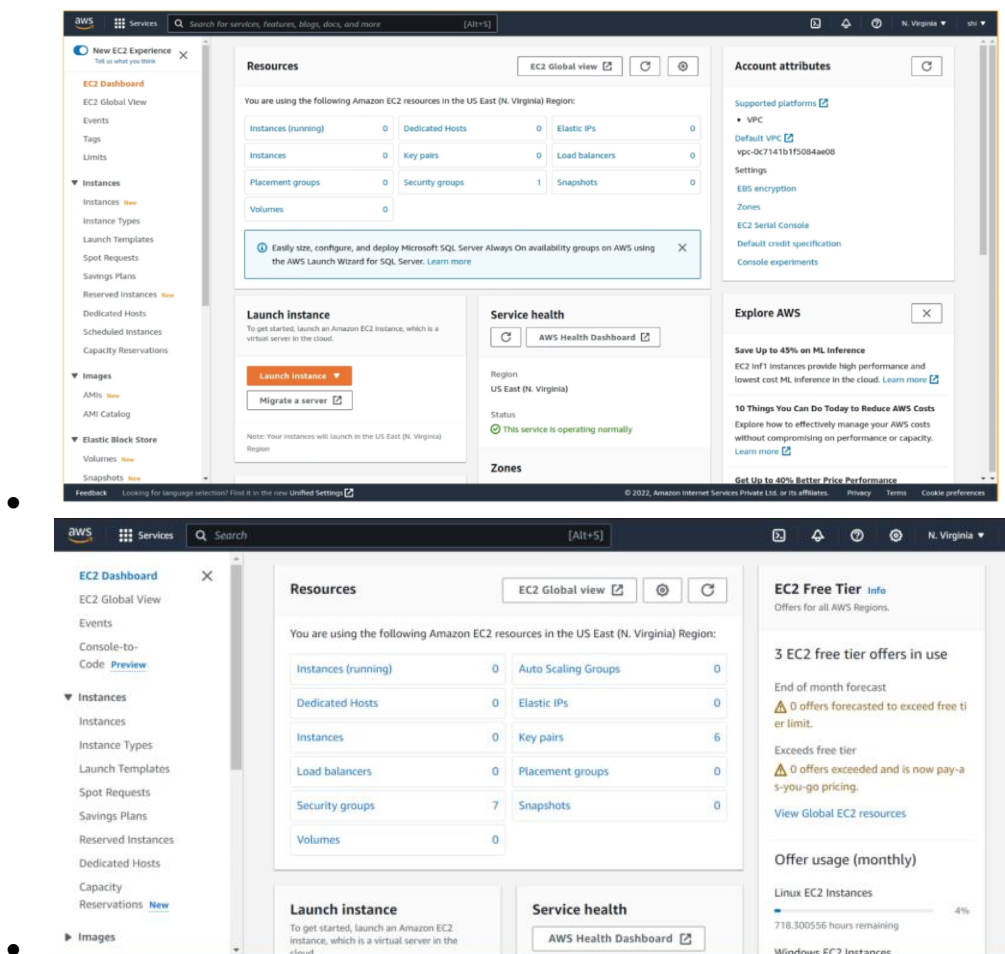
- a. Decide when to **stop or terminate** the instance to avoid unnecessary costs once it’s no longer needed.

If you tell me what kind of workload you're hosting (web app, API, database, etc.), I can tailor these steps into a more specific "golden path" for your environment.

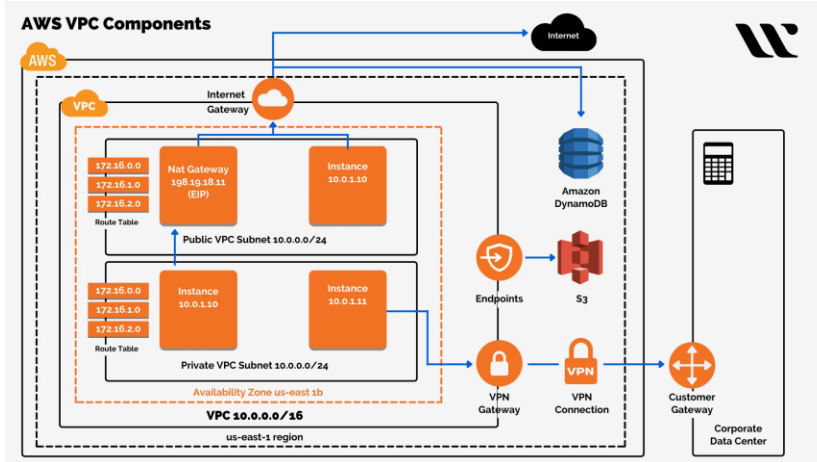
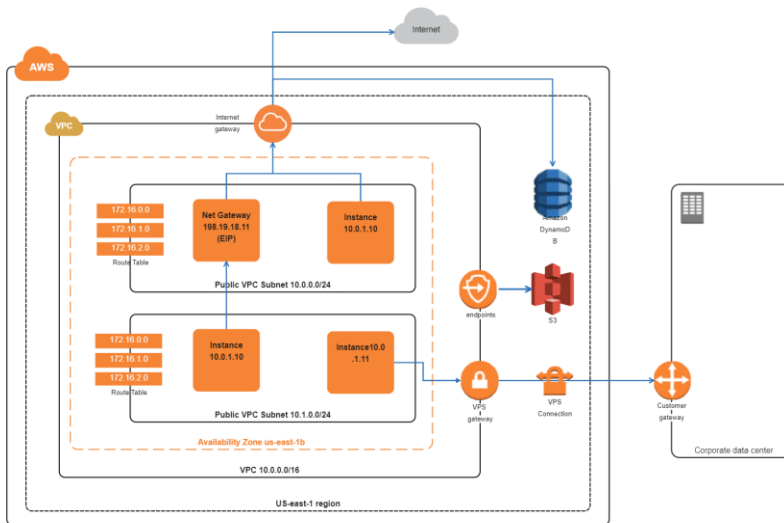
Refined AI Output:

Launching an EC2 Server for a Shopping Web App

A practical, step-by-step guide from the perspective of an AWS Cloud Engineer.



AWS VPC Components



1. Choose the Correct Region

Pick the AWS Region closest to your customers to reduce latency. For a shopping app serving U.S. users, **us-east-1** or **us-east-2** is typical.

- This ensures faster page loads and smoother checkout flows.
- It also helps keep costs predictable and aligns with compliance expectations.

2. Open EC2 and Start the Launch Wizard

Navigate to **EC2 → Launch Instance**. This is where you define the compute environment for your web app.

Use a clear name **such as** **shopping-web-app-prod-01**.

3. Select the AMI (Operating System)

For web apps, I typically choose:

- **Amazon Linux 2** — stable, lightweight, and optimized for AWS
- **Ubuntu LTS** — great if your team prefers Ubuntu tooling

Your AMI should support your stack (Node.js, Python, Java, PHP, etc.).

4. Choose the Instance Type

For a shopping web app, traffic patterns matter:

- **t3.medium** — good baseline for moderate traffic
- **t3.large** — if you expect heavier browsing or flash-sale spikes
- **m5 family** — for more predictable performance

You want enough CPU/RAM to handle product browsing, cart updates, and checkout API calls.

5. Configure Key Pair

Create or select a key pair for SSH access.

- Store the `.pem` file securely.
- Limit access to engineering team members only.

6. Configure Networking (Critical for Web Apps)

This is where shopping apps differ from generic EC2 setups.

VPC & Subnet

- Place the instance in a **public subnet** if it **serves** traffic directly.
- Place it in a **private subnet** if it sits behind a load balancer (recommended for production).

Security Group

Add inbound rules:

- **HTTP (80)** — for web traffic
- **HTTPS (443)** — for secure checkout
- **SSH (22)** — restricted to your office/VPN IP range

This ensures **that** your app is reachable but still protected.

Create Security Group

Security group name ⓘ

Web Server Security Group

Description ⓘ

Security for production web server.

VPC ⓘ

vpc-e68d9c81 | DefaultVPC (default) ▾

Security group rules:

Inbound

Outbound

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ		Description ⓘ
SSH ▾	TCP	22	Anywhere ▾	0.0.0.0/0, ::/0	Admin access.
HTTP ▾	TCP	80	Anywhere ▾	0.0.0.0/0, ::/0	Web traffic.
HTTPS ▾	TCP	443	Custom ▾	0.0.0.0/0, ::/0	Secure web traffic.

Add Rule

<

>

Cancel

Create

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 194.44.90.91/32, My IP for configuration) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Name Info	Description - optional Info
My IP	Add CIDR, prefix list or security 194.44.90.91/32	My IP for configuration

▼ Security group rule 2 (TCP, 22, 52.3.83.134/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.3.83.134/32	data.world inbound connection

▼ Security group rule 3 (TCP, 22, 52.205.195.10/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.205.195.10/32	data.world inbound connection

▼ Security group rule 4 (TCP, 22, 52.205.207.86/32, data.world inbound connection) Remove

Type Info	Protocol Info	Port range Info
ssh	TCP	22
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security 52.205.207.86/32	data.world inbound connection

Add security group rule

7. Configure Storage

Your shopping app will generate logs, images, and cached data.

- Use **gp3** EBS volumes for performance.
- Start with **20–30 GB** for the root volume.
- Add a second volume if your app stores product images locally (though S3 is preferred).

8. Add Tags for Governance

Tags matter for cost tracking and automation:

- Name: *shopping-web-app-prod-01*
- Environment: *production*
- Application: *shopping-web-app*
- Owner: *engineering*

These help with monitoring, billing, and scaling.

9. Review and Launch

Double-check:

- AMI
- Instance type
- Security group
- Storage
- Tags

Then click **Launch Instance**.

10. Connect and Configure the Server

SSH into the instance and install your stack:

- Web server (Nginx or Apache)
- Runtime (Node.js, Python, PHP, Java)
- App code deployment
- Environment variables (DB credentials, API keys)
- SSL certificates (via ACM if using a load balancer)

11. Integrate with Other AWS Services

A shopping app typically uses:

- **RDS or DynamoDB** for product/catalog data
- **S3** for product images

- **CloudFront** for CDN caching
- **ALB** for load balancing
- **CloudWatch** for monitoring
- **IAM roles** for secure access to AWS resources

These services ensure scalability during peak shopping periods.

12. Test the Deployment

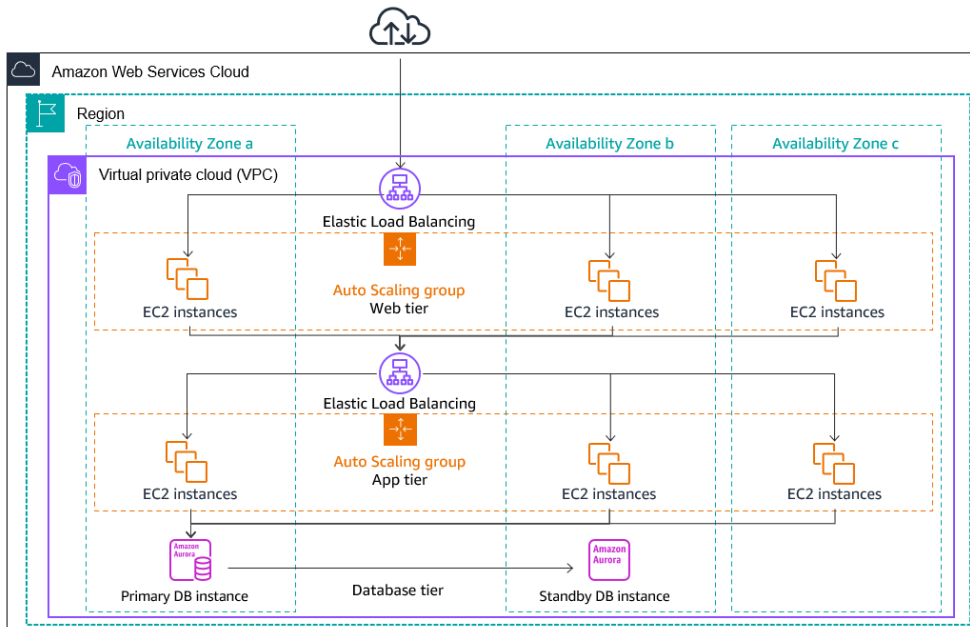
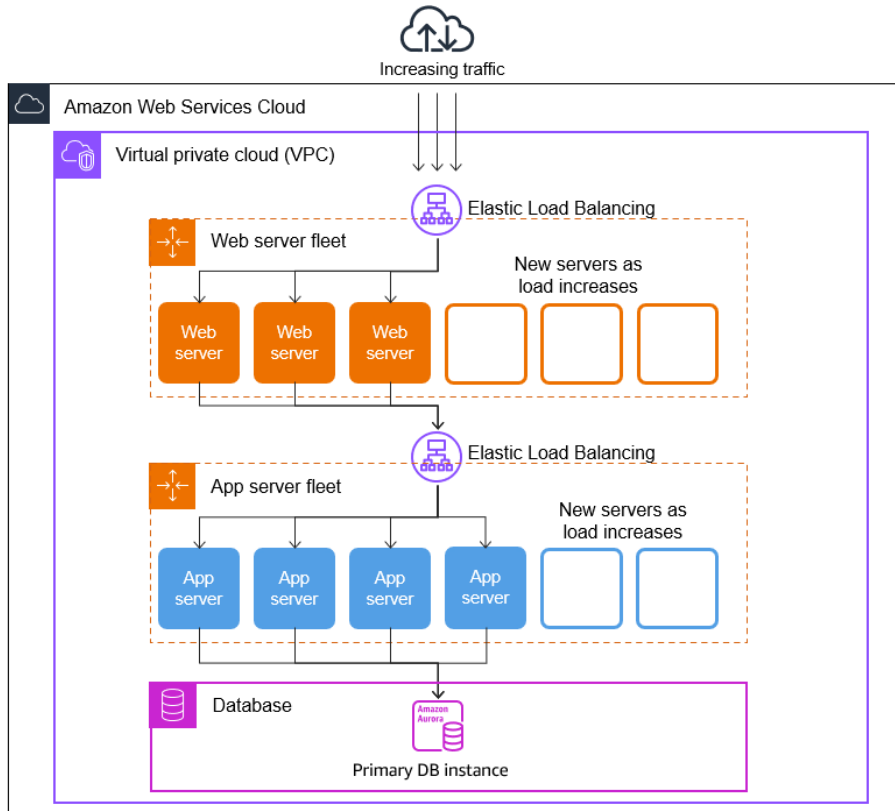
Verify:

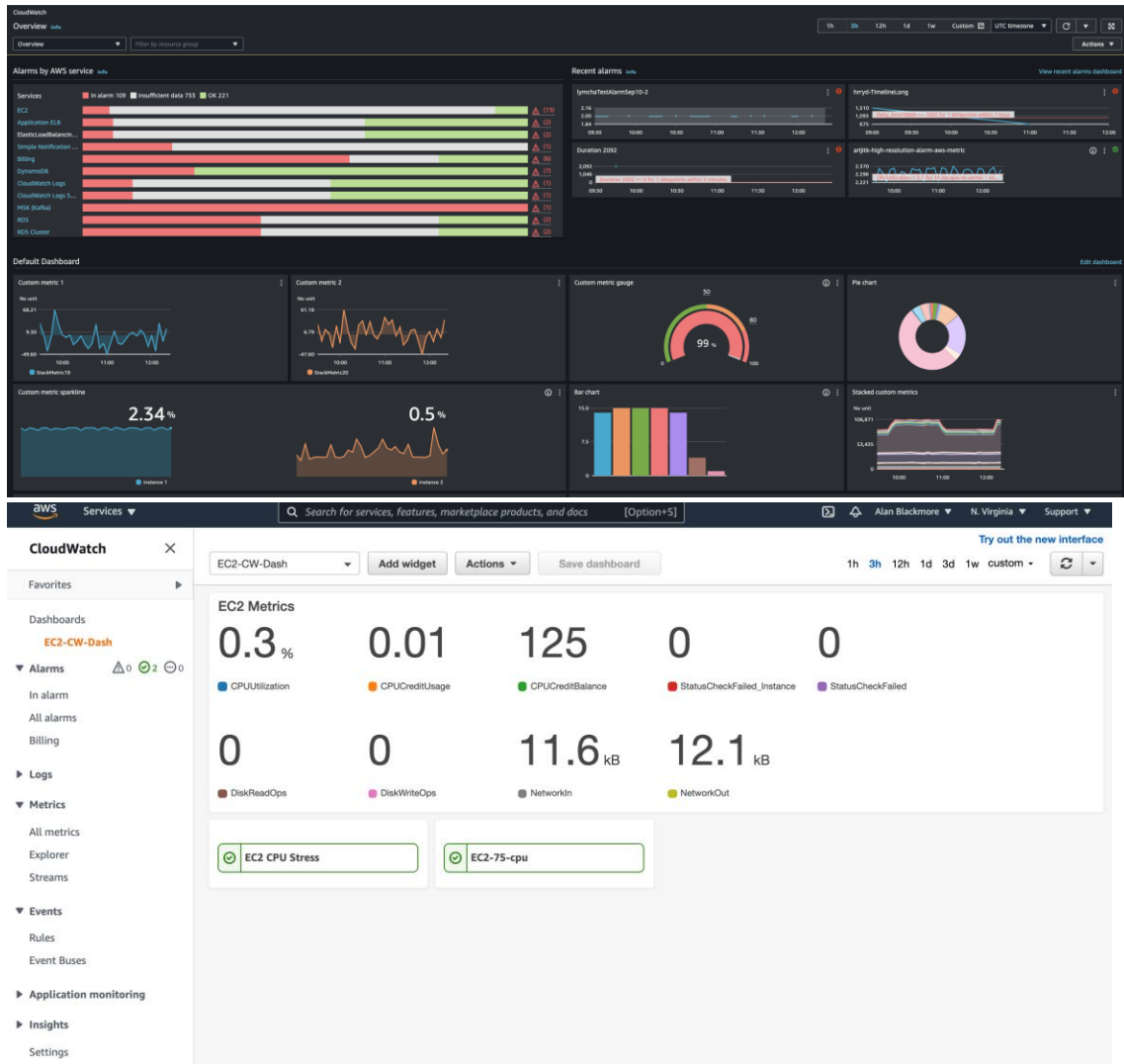
- *Homepage loads*
- *Product pages respond quickly*
- *Cart and checkout APIs work*
- *HTTPS is enforced*
- *Logs are flowing to CloudWatch*

13. Prepare for Scaling

Shopping apps often need elasticity:

- *Configure **Auto Scaling Groups***
- *Add **Load Balancers***
- *Enable **CloudWatch alarms** for CPU, latency, and error rates*





Reflection:

While working with Copilot on this prompt, it showed that a task such as creating a step-by-step guide – while straight forward – can be quite time consuming, especially if their numerous steps in that guide for that task. Copilot made putting together thought processes a lot easier and manageable as it was able to fine tune and put together the steps in a clear and understandable method.

Upon entering my initial prompt, I requested that Copilot get further specific and make a Digital Server for a shopping web app. The follow up prompt was then shown to be even deeper and more in-depth than the first. The main edits I had to make were just simple spacing and grammatical errors. Among some of those edits were just adding prepositions

so that statements flowed better (*This ensures that your app is reachable but still protected*).

I believe that using AI to help put together this guide saved me what would have been at least 3 hours of work by having it list out the steps for me instead of spending that time trying to format and find the right ways to say it. CoPilot also provided sources in which it drew its information from to generate the guide. This allowed for proper checks and proofing to ensure that the information is correct.